Práctica 2: Multiplicación de Karatsuba

Informe técnico · Análisis y Diseño de Algoritmos · Universidad EAFIT · 2026-2
Integrantes: Andrés Osorio, Jerónimo Zapata y Gisel Jaramillo · Repositorio: `ada-practica2-karatsuba`

1. Introducción

Implementamos en C++17 la multiplicación escolar, $\Theta(n^2)$, y la de Karatsuba, $\Theta(n^{\log_2 3})$, para enteros no negativos en base 10 guardados como `std::string` con el dígito más significativo primero. Ambos algoritmos se validaron entre sí y contra `unsigned long long` (20 de 20 pruebas) y luego se midieron para $n = 2, 4, \ldots, 4096$ dígitos. Este informe explica por qué Karatsuba necesita tres productos y no cuatro, resuelve su recurrencia con el Teorema Maestro y con el árbol de recurrencia, y analiza los tiempos. El resultado más llamativo es que **en todo el rango medido la multiplicación clásica fue más rápida**; la sección 4 explica por qué eso no contradice la teoría.

2. La identidad de Karatsuba: tres productos y no cuatro

Sean x e y dos números de n dígitos y $m = \lfloor n/2 \rfloor$. Partimos cada uno en una mitad alta y una baja:

$$x = x_1 \cdot 10^m + x_0, \quad y = y_1 \cdot 10^m + y_0, \quad \text{con } 0 \le x_0, y_0 < 10^m.$$

2.1 El camino directo necesita cuatro productos

Multiplicando término a término:

$$x \cdot y = x_1 y_1 \cdot 10^{2m} + (x_1 y_0 + x_0 y_1) \cdot 10^m + x_0 y_0 \quad (1)$$

Aparecen cuatro productos de números de $\sim n/2$ dígitos: x_1y_1, x_1y_0, x_0y_1 y x_0y_0. Lo demás (sumas y desplazamientos por potencias de 10) cuesta $\Theta(n)$. Dividir y conquistar con esto da $T(n) = 4T(n/2) + \Theta(n)$, cuya solución es $\Theta(n^{\log_2 4}) = \Theta(n^2)$: partir el problema no mejora al método escolar. Para ganar algo hay que reducir el *número* de subproductos.

2.2 Observación clave: solo necesitamos la suma de los productos cruzados

En (1) los productos x_1y_0 y x_0y_1 aparecen siempre sumados; nunca nos hace falta cada uno por separado. Un solo producto de sumas los contiene a ambos:

$$(x_1 + x_0)(y_1 + y_0) = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0$$

Definimos $P_1 = x_1y_1$, $P_2 = x_0y_0$ y $P_3 = (x_1 + x_0)(y_1 + y_0)$. Despejando:

$$x_1y_0 + x_0y_1 = P_3 - P_1 - P_2 \quad (2)$$

y sustituyendo (2) en (1) se obtiene la identidad de Karatsuba:

$$\boldsymbol{x \cdot y = P_1 \cdot 10^{2m} + (P_3 - P_1 - P_2) \cdot 10^m + P_2} \quad (3)$$

Ahora hay solo **tres** multiplicaciones recursivas (P_1, P_2, P_3). A cambio aparecen dos sumas de mitades, dos restas y la recomposición con desplazamientos, todo $\Theta(n)$. Es decir, se cambia una multiplicación (costosa) por un número constante de sumas (baratas). Además, $P_3 - P_1 - P_2 = x_1y_0 + x_0y_1 \ge 0$, así que la resta nunca produce un negativo; el código la hace como `restar(restar(p3, p1), p2)`, que cumple la precondición $a \ge b$ en cada paso.

2.3 Comprobación numérica

Con $x = 1234$ e $y = 5678$, $m = 2$: $x_1 = 12$, $x_0 = 34$, $y_1 = 56$, $y_0 = 78$. Entonces $P_1 = 12 \cdot 56 = 672$, $P_2 = 34 \cdot 78 = 2652$ y $P_3 = (12+34)(56+78) = 46 \cdot 134 = 6164$. El término central es $6164 - 672 - 2652 = 2840$, y el producto es $672 \cdot 10^4 + 2840 \cdot 10^2 + 2652 = 6\,720\,000 + 284\,000 + 2\,652 = \mathbf{7\,006\,652} = 1234 \cdot 5678$.

2.4 Detalle que afecta la implementación

Las sumas $x_1 + x_0$ y $y_1 + y_0$ pueden tener un dígito más (acarreo), de modo que P_3 multiplica números de hasta $\lceil n/2 \rceil + 1$ dígitos y no exactamente $n/2$. Esto no cambia el orden asintótico, pero obliga a cuidar la terminación: para $n = 3$ ese tamaño sigue siendo 3 y la recursión no avanzaría, mientras que para $n \geq 4$ el tamaño es estrictamente menor que n. Por eso el caso base del código usa la multiplicación clásica cuando $n \leq 3$ (constante `UMBRAL = 3`). Además, `multiplicarKaratsuba` rellena ambos números con ceros a la izquierda hasta una potencia de 2, para que las mitades queden balanceadas.

3. La recurrencia y su solución por dos métodos

Con la identidad (3), el costo de multiplicar dos números de n dígitos satisface (ignorando pisos, techos y el dígito extra de las sumas, que no alteran el orden asintótico):

$$T(n) = 3T(n/2) + cn, \quad T(1) = \Theta(1)$$

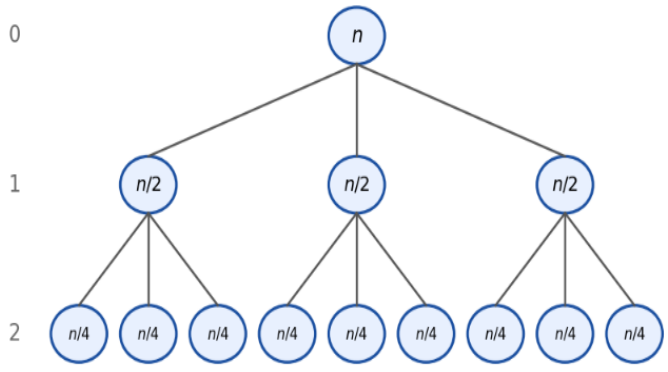
El término cn reúne el trabajo lineal de cada llamada: partir los números, sumar mitades, restar, desplazar y recomponer.

3.1 Teorema Maestro

La recurrencia tiene la forma $T(n) = aT(n/b) + f(n)$ con $\boldsymbol{a = 3}$, $\boldsymbol{b = 2}$ y $\boldsymbol{f(n) = cn = \Theta(n)}$. El valor crítico es $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$. Comparamos: $f(n) = n^1$ y $1 < \log_2 3 \approx 1.585$. Tomando $\varepsilon = \log_2 3 - 1 \approx 0.585 > 0$ se cumple $f(n) = O(n^{\log_b a - \varepsilon})$, es decir, f crece polinomialmente *más despacio* que el valor crítico. Esto es el **caso 1** y, por lo tanto, $T(n) = \Theta(n^{\log_2 3})$ (el caso 1 no exige condición de regularidad). Como contraste, con cuatro productos sería $a = 4$, valor crítico n^2 y $f(n) = O(n^{2-1})$: también caso 1, pero con resultado $\Theta(n^2)$. Pasar de 4 a 3 subproblemas es lo que baja el exponente de 2 a 1.585.

3.2 Árbol de recurrencia

El árbol muestra *dónde* se gasta el trabajo. En el nivel i hay 3^i subproblemas, cada uno de tamaño $n/2^i$ y con costo $c \cdot n/2^i$, de modo que el costo del nivel es $3^i \cdot c \cdot n/2^i = (3/2)^i \cdot cn$. El tamaño se reduce a 1 cuando $n/2^h = 1$, es decir, la **altura** es $h = \log_2 n$ (el árbol tiene $h + 1$ niveles).



nivel	subproblemas	tamaño	costo del nivel
0	1	n	cn
1	3	$n/2$	$(3/2)\,cn$
2	9	$n/4$	$(3/2)^2\,cn$
i	3^i	$n/2^i$	$(3/2)^i\,cn$
h	hojas: tamaño 1, costo $\Theta(1)$ cada una	$3^{\log_2 n} = n^{\log_2 3}$	1	$\Theta(n^{\log_2 3})$

Hojas. En el último nivel hay $3^h = 3^{\log_2 n}$ subproblemas. Como $3 = 2^{\log_2 3}$, resulta $3^{\log_2 n} = (2^{\log_2 n})^{\log_2 3} = n^{\log_2 3}$ hojas, cada una con costo $\Theta(1)$.

Niveles internos ($i = 0, \ldots, h-1$): serie geométrica de razón $3/2$:

$$\Sigma_{i=0}^{h-1} (3/2)^i \cdot cn = cn \cdot [(3/2)^h - 1]/(3/2 - 1) = 2cn \cdot [(3/2)^h - 1]$$

y como $(3/2)^h = 3^h/2^h = n^{\log_2 3}/n$, queda $2c(n^{\log_2 3} - n)$.

Suma total. $T(n) = 2c(n^{\log_2 3} - n) + \Theta(n^{\log_2 3}) = \Theta(n^{\log_2 3})$.

Por qué el exponente es $\log_2 3$. La razón $3/2$ es mayor que 1, así que el costo por nivel *crece* hacia abajo y el último nivel domina la suma; por eso el orden total coincide con el número de hojas. Ese número es 3 elevado a la cantidad de veces que se puede partir n por la mitad ($3^{\log_2 n}$), y escrito como potencia de n da exactamente $n^{\log_2 3}$. Con cuatro subproblemas serían $4^{\log_2 n} = n^2$ hojas.

Qué aporta cada método. El Teorema Maestro da el veredicto rápido y obliga a justificar el caso comparando $f(n)$ con $n^{\log_2 3}$. El árbol explica el *porqué*: muestra que el trabajo se concentra en las hojas, que el costo por nivel crece con razón $3/2$ y que el exponente sale de contar hojas; ambos coinciden en $\Theta(n^{\log_2 3})$.

4. Análisis experimental

4.1 Montaje

- **Código medido.** Clásica: acumula los productos de dígitos en un `vector<int>`, omite las filas de dígito cero y propaga los acarreos una sola vez al final. Karatsuba: recursiva según la identidad (3), caso base clásico para $n \leq 3$ y relleno a potencia de 2.
- **Entradas.** Dos números aleatorios de n dígitos (primer dígito distinto de cero), generados con `mt19937_64` con semilla 42, para $n = 2^k$, $k = 1, \ldots, 12$. Con n potencia de 2, Karatsuba no necesita relleno.
- **Control de correctitud.** Antes de medir cada n se verifica que ambos algoritmos producen el mismo producto.
- **Medición.** `std::chrono::steady_clock`; una ejecución de calentamiento; luego 5 mediciones, cada una repite la multiplicación hasta acumular al menos 100 ms y reporta µs por llamada. Se usa la **mediana** de las 5 (el promedio queda en el CSV). El resultado se consume en una variable `volatile` para que el compilador no elimine el cálculo. Compilación: `g++ -std=c++17 -O2`.
- **Alcance.** Medido en un PC con Windows. Los tiempos absolutos dependen del equipo; se interpretan razones y pendientes. Un solo par de números por tamaño, así que hay algo de ruido.

n (dígitos)	Clásica (µs)	Karatsuba (µs)	Clásica / Karatsuba	Karatsuba es … veces más lenta
2	0.450	0.618	0.729	1.4
4	0.627	3.005	0.209	4.8
8	0.866	8.093	0.107	9.3
16	1.29	27.867	0.046	21.6
32	2.25	94.952	0.024	42.1
64	4.22	305.281	0.014	72.3
128	11.6	941.044	0.012	81.1
256	40.4	2924.486	0.014	72.4
512	159	8848.683	0.018	55.7
1024	593	25768.475	0.023	43.4
2048	2 313	90310.400	0.026	39.0
4096	9 414	217068.900	0.043	23.1

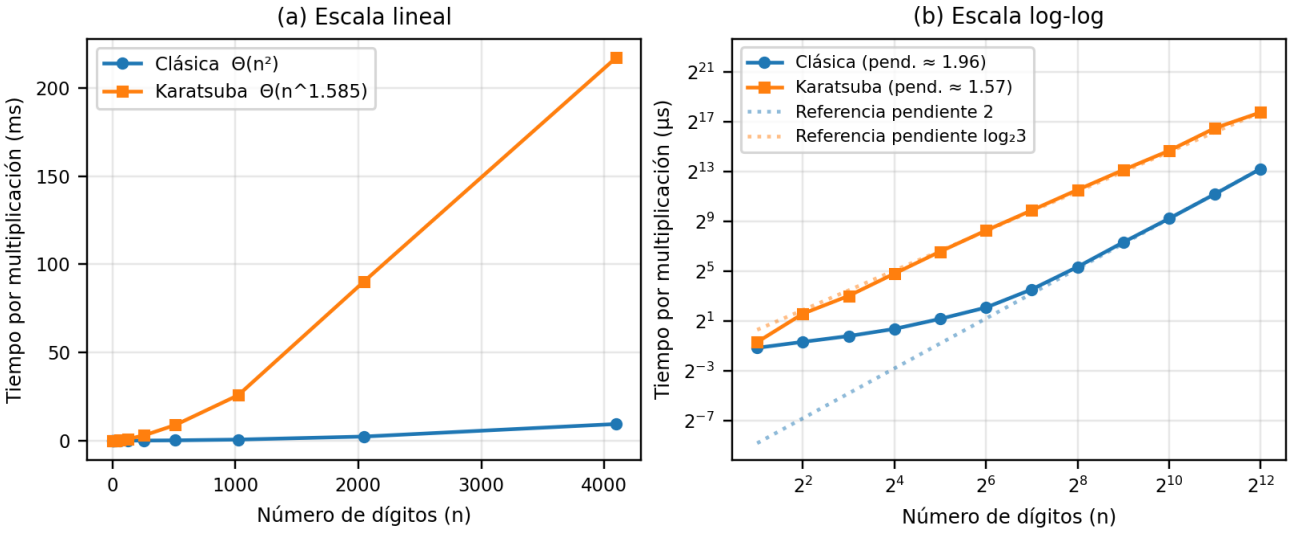


Figura 2. Tiempos medidos (mediana) en escala lineal (a) y log-log (b). Las rectas punteadas tienen las pendientes teóricas 2 y $\log_2 3 \approx 1.585$. Gráficas generadas con `datos/graficar.py` *y rehechas con el mismo CSV.*

La pendiente medida con los últimos cuatro puntos (n = 512 a 4096) es **1.96** para la clásica y **1.57** para Karatsuba, frente a los valores teóricos 2 y 1.585. Además, desde n = 16 el tiempo de Karatsuba se multiplica en promedio por ≈ 3.07 al duplicar n (con ruido en pasos individuales, entre 2.4 y 3.5), que es lo que predice $3T(n/2)$; el de la clásica se acerca a 4 (3.5, 3.9, 3.7, 3.9 y 4.1 en los últimos cinco duplicados). La teoría describe bien la *forma* de las curvas. En escala lineal (Figura 2a) la clásica casi no se despega del eje mientras Karatsuba llega a ≈ 217 ms en n = 4096, lo que resume el resultado: en este rango el método de menor orden asintótico es el más lento.

4.3 Interpretación del punto de cruce

No hay cruce en el rango 2 … 4096: la clásica es más rápida en todos los tamaños. En n = 4096 tarda 9.41 ms frente a 217 ms de Karatsuba, es decir, Karatsuba es unas 23 veces más lenta. Sin embargo, la razón clásica/Karatsuba no es monótona (Figura 3): cae desde 0.73 en n = 2 hasta un mínimo de 0.0123 en n = 128 (donde Karatsuba es ≈ 81 veces más lenta) y desde ahí *sube* en cada duplicación (0.0138, 0.0180, 0.0230, 0.0256, 0.0434).

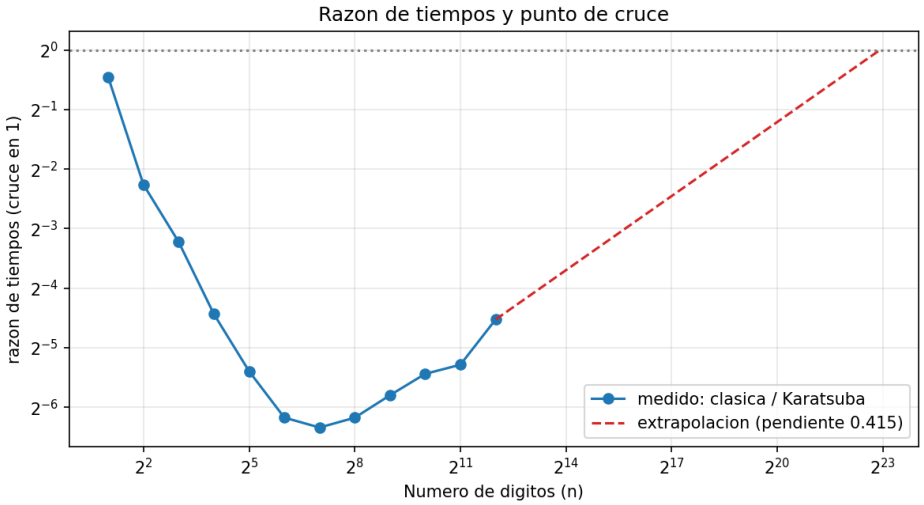


Figura 3. Razón de tiempos clásica/Karatsuba (la igualdad es el cruce) y extrapolación por ley de potencias desde n = *4096.*

La caída inicial se explica porque, para n pequeño, el tiempo de la clásica está dominado por costos fijos (reservar vectores, normalizar) y crece mucho menos que ×4 al duplicar n (≈ ×1.6 en promedio hasta n = 64), mientras que Karatsuba ya crece ≈ ×3.5. A partir de $n \approx 128$ el término cuadrático de la clásica empieza a mandar (≈ ×3.5 → ×4) y la razón sube; su crecimiento por duplicación (×1.12, 1.30, 1.28, 1.11, 1.69; ≈ ×1.29 en promedio) se acerca al valor teórico $2^{2-\log_2 3} \approx 1.33$. El último salto (×1.69) es mayor que lo esperado; con un solo par de números por tamaño puede deberse en parte a ruido de medición o a efectos de caché, pero no lo verificamos. Si la tendencia continúa, la razón llega a 1 cuando $n \approx 4096 \cdot (1/0.0434)^{1/0.415} \approx 8\times10^6$ dígitos (≈ 7×10^6 usando la pendiente medida 0.42 en lugar de la teórica 0.415). Es una **extrapolación**, no una medición: a esos tamaños la memoria y la caché cambiarían las constantes, pero indica que el cruce de *esta* implementación está varios órdenes de magnitud más allá de lo medido.

4.4 Complejidad asintótica frente a rendimiento real

5. Conclusiones y dificultades

5.1 Conclusiones

- La identidad (3) cambia un producto por sumas lineales, y eso baja la recurrencia de $4T(n/2) + \Theta(n)$ a $3T(n/2) + \Theta(n)$. Ambos métodos (Maestro, caso 1; árbol, $3^{\log_2 n}$ hojas) dan $\Theta(n^{\log_2 3})$.
- Los experimentos confirman la forma: pendiente 1.57 para Karatsuba y 1.96 para la clásica, y un factor ≈ 3 por duplicación de n en Karatsuba.
- No hubo punto de cruce entre 2 y 4096 dígitos; la clásica fue hasta ≈ 81 veces más rápida (en n = 128) y unas 23 veces en n = 4096. La razón se acerca lentamente a 1, con un cruce extrapolado del orden de 10^7 dígitos (≈ 7×10^6 a 8×10^6).
- Menor complejidad asintótica no implica menor tiempo en el rango práctico. Las constantes, las copias y las asignaciones de memoria pueden dominar durante muchos órdenes de magnitud.
- Mejoras que probaríamos (no medidas en este trabajo): un umbral mayor para cambiar a la clásica, representar los números en una base mayor (por ejemplo 10^9) en vectores de enteros y evitar las copias de `normalizar` y `partir` trabajando con índices.

5.2 Dificultades encontradas

- **Terminación de la recursión:** el dígito extra de las sumas de mitades hace que para n = 3 el subproblema no se reduzca; se resolvió con el caso base `UMBRAL = 3`.
- **Restas sin negativos:** el orden $(P_3 - P_1) - P_2$ y la precondición $a \geq b$ de `restar` (con apoyo de `comparar`) evitan aritmética con signo.
- **Ceros a la izquierda y longitudes distintas o impares:** `partir` conserva ceros en las mitades, por lo que se normalizan antes de recursar; la función principal rellena a potencia de 2.
- **Medición confiable:** evitar que el compilador elimine el cálculo (`volatile`), tiempos demasiado cortos para el reloj (repetir hasta 100 ms), ruido (mediana de 5 y calentamiento) y verificar la igualdad de resultados antes de medir.
- **Interpretar la ausencia de cruce:** distinguir la complejidad asintótica del rendimiento real, apoyándonos en las pendientes, la razón de tiempos y el conteo de llamadas y copias.

6. Aportes individuales

- **Andrés Osorio:** Implementación de los archivos base, algoritmos de clásica y Karatsuba, una prueba temporal y funciones auxiliares.
- **Jerónimo Zapata:** Creación del repositorio, realización de los tests, casos de borde, productos grandes y pruebas aleatorias, la versión final de la validación, medicion de tiempos clasica vs karatsuba, tiempos medidos y gráficas y organización del README.
- **Gisel Jaramillo:** revisión del código y verificación de las pruebas, comparación de resultados y redacción del informe.

7. Declaración de uso de inteligencia artificial

Se utilizó **ChatGPT y Gemini** principalmente como apoyo para entender mejor el tema y para pedir sugerencias para realizar un mejor diseño y estructuración del código base que se creó inicialmente y los que se fueron construyendo durante la realización del proyecto.

Dado a que tanto las preguntas, como las respuestas recibidas fueron en su mayoría bastante extensas, a continuación se pondrá el link del chat completo que se tuvo con esta IA:

- Chat con ChatGPT: https://chatgpt.com/share/6ae419d5-e6c0-83e9-b8f2-a3b6ea1b1260
- Chat de Gemini: https://share.gemini.google/gKKktgO0W6z3